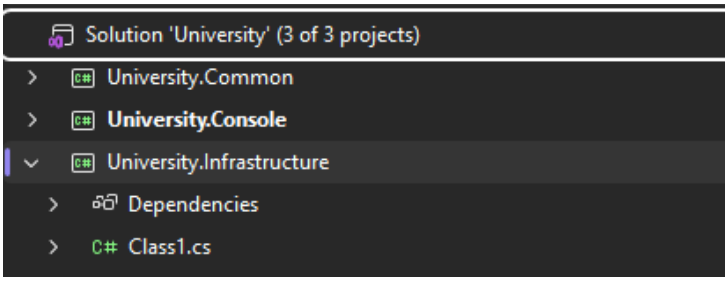
Звіт з лабораторної роботи №3

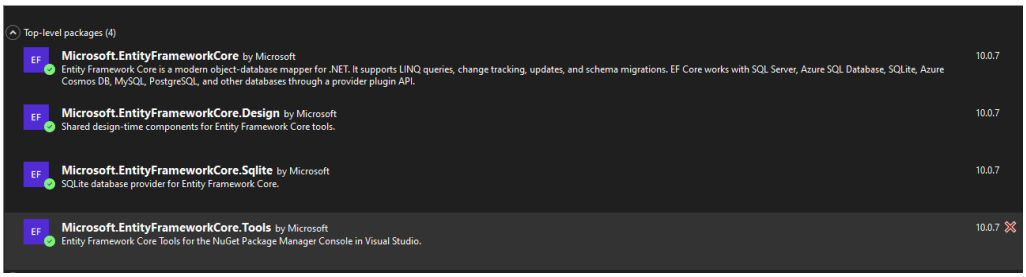
Дорошенко 301-ТК

Робота із базами даних в .NET. Використання бібліотеки Entity Framework.

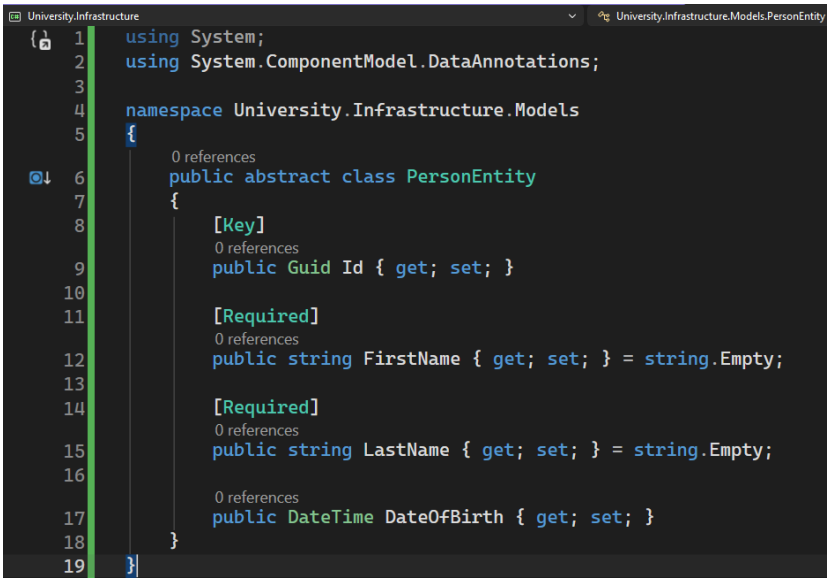
Створюю новий проєкт у існуючому рішенні з минулих робіт:



Встановлюю необхідні NuGet пакети:



Створюю EntityModels на прикладі PersonEntity.cs:



```
University.Infrastructure                    University.Infrastructure.Models.PersonEntity
1   using System;
2   using System.ComponentModel.DataAnnotations;
3
4   namespace University.Infrastructure.Models
5   {
        0 references
6       public abstract class PersonEntity
7       {
8           [Key]
            0 references
9           public Guid Id { get; set; }
10
11          [Required]
            0 references
12          public string FirstName { get; set; } = string.Empty;
13
14          [Required]
            0 references
15          public string LastName { get; set; } = string.Empty;
16
            0 references
17          public DateTime DateOfBirth { get; set; }
18      }
19  }
```

Створюю UniversityContext:

CrudServiceAsync.cs | ICrudServiceAsync.cs | StudentEntity.cs | **UniversityContext.cs** | CourseEntity.cs

University.Infrastructure | University.Infrastruc

```
using Microsoft.EntityFrameworkCore;
using University.Infrastructure.Models;

namespace University.Infrastructure.Data
{
    2 references
    public class UniversityContext : DbContext
    {
        0 references
        public DbSet<StudentEntity> Students { get; set; }
        0 references
        public DbSet<TeacherEntity> Teachers { get; set; }
        0 references
        public DbSet<CourseEntity> Courses { get; set; }

        0 references
        public UniversityContext(DbContextOptions<UniversityContext> options)
            : base(options)
        {
        }

        0 references
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            base.OnModelCreating(modelBuilder);

            // Table-per-Type (TPT) успадкування
            modelBuilder.Entity<PersonEntity>()
                        .UseTptMappingStrategy();

            modelBuilder.Entity<StudentEntity>()
                        .ToTable("Students");

            modelBuilder.Entity<TeacherEntity>()
                        .ToTable("Teachers");

            // Зв'язки
            modelBuilder.Entity<CourseEntity>()
                        .HasOne(c => c.Teacher)
                        .WithMany()
                        .HasForeignKey(c => c.TeacherId)
                        .OnDelete(DeleteBehavior.SetNull);

            // Додаткові налаштування
            modelBuilder.Entity<TeacherEntity>()
                        .Property(t => t.Salary)
                        .HasColumnType("decimal(18,2)");
        }
    }
}
```

Створюю Repository та реалізую його у проєкті University.Infrastructure.

```
Repository.cs   IRepository.cs   CrudServiceAsync.cs   ICrudServiceAsync.cs   StudentEntity.cs   Univ
University.Infrastructure                                                    University.Infras
```

```
using System;
using System.Collections.Generic;
using System.Threading.Tasks;

namespace University.Infrastructure.Repositories
{
    1 reference
    public interface IRepository<T> where T : class
    {
        1 reference
        Task<T?> GetByIdAsync(Guid id);
        1 reference
        Task<IEnumerable<T>> GetAllAsync();
        1 reference
        Task AddAsync(T entity);
        1 reference
        Task UpdateAsync(T entity);
        1 reference
        Task DeleteAsync(T entity);
        1 reference
        Task SaveChangesAsync();
    }
}
```

```
Repository.cs   IRepository.cs   CrudServiceAsync.cs   ICrudServiceAsync.cs
University.Infrastructure
```

```
using System.Threading.Tasks;
using Microsoft.EntityFrameworkCore;
using University.Infrastructure.Data;

namespace University.Infrastructure.Repositories
{
    1 reference
    public class Repository<T> : IRepository<T> where T : class
    {
        protected readonly UniversityContext _context;
        protected readonly DbSet<T> _dbSet;

        0 references
        public Repository(UniversityContext context)
        {
            _context = context;
            _dbSet = context.Set<T>();
        }

        1 reference
        public async Task<T?> GetByIdAsync(Guid id)
        {
            return await _dbSet.FindAsync(id);
        }

        1 reference
        public async Task<IEnumerable<T>> GetAllAsync()
        {
            return await _dbSet.ToListAsync();
        }

        1 reference
        public async Task AddAsync(T entity)
        {
            await _dbSet.AddAsync(entity);
        }

        1 reference
        public Task UpdateAsync(T entity)
        {
            _dbSet.Update(entity);
            return Task.CompletedTask;
        }

        1 reference
        public Task DeleteAsync(T entity)
        {
            _dbSet.Remove(entity);
            return Task.CompletedTask;
        }

        1 reference
        public async Task SaveChangesAsync()
        {
            await _context.SaveChangesAsync();
        }
    }
}
```

Оновлюю CrudServiceAsync під программу:

Repository.cs | IRepository.cs | CrudServiceAsync.cs* | ICrudServiceAsync.cs | StudentEntity.cs | UniversityCo

University.Common | University.Infrastructure.S

```
using Microsoft.EntityFrameworkCore;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Linq;
using System.Text.Json;
using System.Threading.Tasks;
using University.Common.Services;
using University.Infrastructure.Data;
using University.Infrastructure.Repositories;

namespace University.Infrastructure.Services
{
    1 reference
    public class CrudServiceAsync<T> : ICrudServiceAsync<T> where T : class
    {
        private readonly IRepository<T> _repository;
        private readonly string _filePath; // для резервного збереження

        0 references
        public CrudServiceAsync(IRepository<T> repository, string filePath = null)
        {
            _repository = repository;
            _filePath = filePath ?? $"backup_{typeof(T).Name}.json";
        }

        1 reference
        public async Task<bool> CreateAsync(T element)
        {
            if (element == null) return false;
            await _repository.AddAsync(element);
            await _repository.SaveChangesAsync();
            Console.WriteLine($"[+] Асинхронно створено {typeof(T).Name} в БД");
            return true;
        }

        1 reference
        public async Task<T?> ReadAsync(Guid id)
        {
            return await _repository.GetByIdAsync(id);
        }

        4 references
        public async Task<IEnumerable<T>> ReadAllAsync()
        {
            return await _repository.GetAllAsync();
        }

        1 reference
        public async Task<IEnumerable<T>> ReadAllAsync(int page, int amount)
        {
            var all = await ReadAllAsync();
            return all.Skip(page * amount).Take(amount);
        }


        public async Task<bool> UpdateAsync(T element)
        {
            if (element == null) return false;
            await _repository.UpdateAsync(element);
            await _repository.SaveChangesAsync();
            return true;
        }

        1 reference
        public async Task<bool> RemoveAsync(T element)
        {
            if (element == null) return false;
            await _repository.DeleteAsync(element);
            await _repository.SaveChangesAsync();
            return true;
        }

        1 reference
        public async Task<bool> SaveAsync()
        {
            // Додаткове збереження у JSON як backup
            try
            {
                var all = await ReadAllAsync();
                string json = JsonSerializer.Serialize(all, new JsonSerializerOptions { WriteIndented = true });
                await File.WriteAllTextAsync(_filePath, json);
                Console.WriteLine($"[💾] Backup збережено у {_filePath}");
                return true;
            }
            catch
            {
                return false;
            }
        }

        1 reference
        public IEnumerator<T> GetEnumerator() => ReadAllAsync().Result.GetEnumerator();
        0 references
        IEnumerator IEnumerable.GetEnumerator() => GetEnumerator();
    }
}
```

Додаю міграцію Entity Framework:

```
Package Manager Console
Package source: All        Default project: University.Infrastructure
PM> Add-Migration InitialCreate
Build started...
Build succeeded.
To undo this action, use Remove-Migration.
PM> Update-Database
Build started...
Build succeeded.
Acquiring an exclusive lock for migration application. See https://aka.ms/efcore-docs-migrations-lock for more information if this takes too long.
Applying migration '20260512124631_InitialCreate'.
Done.
PM>
```

Реалізація програми:

```
=== Лабораторна робота №3: Entity Framework Core ===

Створення 50 студентів у базі даних...
=== Клас Person був ініціалізований ===
[+] Асинхронно створено StudentEntity в БД
[+] Асинхронно створено StudentEntity в БД
[+] Асинхронно створено StudentEntity в БД
[+] Асинхронно створено StudentEntity в БД
[+] Асинхронно створено StudentEntity в БД
[+] Асинхронно створено StudentEntity в БД
[+] Асинхронно створено StudentEntity в БД
[+] Асинхронно створено StudentEntity в БД
[+] Асинхронно створено StudentEntity в БД
[+] Асинхронно створено StudentEntity в БД
[+] Асинхронно створено StudentEntity в БД
[+] Асинхронно створено StudentEntity в БД
[+] Асинхронно створено StudentEntity в БД
[+] Асинхронно створено StudentEntity в БД
[+] Асинхронно створено StudentEntity в БД
[+] Асинхронно створено StudentEntity в БД
[+] Асинхронно створено StudentEntity в БД
[+] Асинхронно створено StudentEntity в БД
[+] Асинхронно створено StudentEntity в БД
[+] Асинхронно створено StudentEntity в БД
[+] Асинхронно створено StudentEntity в БД
[+] Асинхронно створено StudentEntity в БД
[+] Асинхронно створено StudentEntity в БД
[+] Асинхронно створено StudentEntity в БД
[+] Асинхронно створено StudentEntity в БД
[+] Асинхронно створено StudentEntity в БД
[+] Асинхронно створено StudentEntity в БД
[+] Асинхронно створено StudentEntity в БД
[+] Асинхронно створено StudentEntity в БД
[+] Асинхронно створено StudentEntity в БД
[+] Асинхронно створено StudentEntity в БД
[+] Асинхронно створено StudentEntity в БД
[+] Асинхронно створено StudentEntity в БД
[+] Асинхронно створено StudentEntity в БД
[+] Асинхронно створено StudentEntity в БД
[+] Асинхронно створено StudentEntity в БД
[+] Асинхронно створено StudentEntity в БД
[+] Асинхронно створено StudentEntity в БД
[+] Асинхронно створено StudentEntity в БД
[+] Асинхронно створено StudentEntity в БД
[+] Асинхронно створено StudentEntity в БД
[+] Асинхронно створено StudentEntity в БД
[+] Асинхронно створено StudentEntity в БД
[+] Асинхронно створено StudentEntity в БД
[+] Асинхронно створено StudentEntity в БД
[+] Асинхронно створено StudentEntity в БД
[+] Асинхронно створено StudentEntity в БД
[+] Асинхронно створено StudentEntity в БД
[+] Асинхронно створено StudentEntity в БД
[+] Асинхронно створено StudentEntity в БД
[+] Асинхронно створено StudentEntity в БД
Час виконання: 195 мс
Студентів у БД: 50
Середній GPA: 4.05

=== Лабораторна №3 виконана! ===
```

Лістинг програми:

```
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.DependencyInjection;
using System;
using System.Diagnostics;
using System.Linq;
using System.Threading.Tasks;
using University.Common.Models;
using University.Common.Services;
using University.Infrastructure.Data;
using University.Infrastructure.Models;
using University.Infrastructure.Repositories;
using University.Infrastructure.Services;

namespace University.ConsoleApp
{
  internal class Program
  {
    static async Task Main(string[] args)
    {
      Console.OutputEncoding = System.Text.Encoding.UTF8;

      Console.WriteLine("=== Лабораторна робота №3: Entity Framework Core ===\n");

      // Налаштування DI та DbContext
      var services = new ServiceCollection();

      services.AddDbContext<UniversityContext>(options =>
        options.UseSqlite("Data Source=university.db"));

      services.AddScoped(typeof(IRepository<>), typeof(Repository<>));
      services.AddScoped(typeof(ICrudServiceAsync<>), typeof(CrudServiceAsync<>));

      var serviceProvider = services.BuildServiceProvider();

      using var scope = serviceProvider.CreateScope();
      var context = scope.ServiceProvider.GetRequiredService<UniversityContext>();
      await context.Database.MigrateAsync(); // застосовуємо міграції

      var studentRepo = scope.ServiceProvider.GetRequiredService<IRepository<StudentEntity>>();
      var studentService = new CrudServiceAsync<StudentEntity>(studentRepo);

      // === Тестування ===
      var stopwatch = Stopwatch.StartNew();

      Console.WriteLine("Створення 50 студентів у базі даних...");
```

```
        for (int i = 0; i < 50; i++)
        {
            var student = Student.CreateNew(); // з Common

            var studentEntity = new StudentEntity
            {
                Id = student.Id,
                FirstName = student.FirstName,
                LastName = student.LastName,
                DateOfBirth = student.DateOfBirth,
                StudentId = student.StudentId,
                Course = student.Course,
                GPA = student.GPA
            };

            await studentService.CreateAsync(studentEntity);
        }

        stopwatch.Stop();

        var allStudents = (await studentService.ReadAllAsync()).ToList();

        Console.WriteLine($"Час виконання: {stopwatch.ElapsedMilliseconds} мс");
        Console.WriteLine($"Студентів у БД: {allStudents.Count}");
        Console.WriteLine($"Середній GPA: {allStudents.Average(s => s.GPA):F2}");

        Console.WriteLine("\n=== Лабораторна №3 виконана! ===");
        Console.ReadKey();
    }
  }
}
```